

STUDY GUIDE

PROJECT DEFENSE MASTER GUIDE

Cairo Metro Route Planner
ASP.NET Core MVC — Complete Viva Preparation

16 Comprehensive Sections • 100+ Exam Questions • Arabic & English

Table of Contents

1	Project Overview	
2	MVC Architecture	
3	Database Analysis	
4	DbContext Analysis	
5	Controller Defense Guide	
5.1		RoutesController
5.2		HomeController
6	Program.cs Defense	
7	Dependency Injection	
8	Entity Framework Core	
9	LINQ Analysis	
10	Validation & Data Annotations	
11	Security	
12	Razor & Views	
13	Project Flow Walkthrough	
14	احكيلي المشروع كآني بشرحه للدكتور	
15	مراجعة ليلة المناقشة	

PROJECT DEFENSE MASTER GUIDE — CAIRO METRO

كل اللي محتاج تعرفه عن المشروع موجود هنا — بالعربي. **(Viva) الملف ده هيساعدك تستعد لمناقشة المشروع** المصري مع المصطلحات التقنية بالإنجليزي.

1. PROJECT OVERVIEW

Technical Content

1.1 Problem Statement

The Cairo Metro system has 3 lines, 89 stations, and multiple intersecting transfer points. Passengers need to find the **shortest path** between any two stations, know the **fare price**, **estimated travel time**, and **number of transfers**. This system solves that problem with an interactive web application.

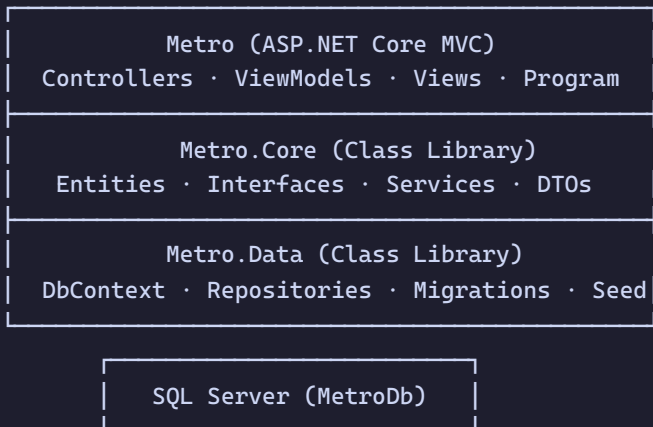
1.2 Target Users

- Cairo Metro passengers planning trips
- Tourists navigating the metro system
- Anyone needing fare/time/transfer estimates

1.3 Main Features

Feature	Description
Shortest Path Calculation	Dijkstra's algorithm on a weighted graph
Fare Calculation	Tiered pricing based on station count
Travel Time Estimate	2 minutes per station
Transfer Detection	Counts line changes along the path
Interactive Map	Leaflet + OpenStreetMap with GPS location
Searchable Dropdowns	Tom Select library for station selection
GPS Nearest Station	Finds closest station using Haversine formula

1.4 System Architecture (3-Tier)



1.5 Request Lifecycle

```

Browser → POST /Routes/Index
→ Routing maps to RoutesController.Index
→ Controller calls _metroService.GetRouteAsync(1, 19)
→ MetroService calls RouteService → GraphBuilder → Repositories → DbContext
→ Dijkstra algorithm runs → RouteResultDto returned
→ Controller maps to ViewModel → Returns View
→ Razor renders Index.cshtml → HTML sent to browser
  
```

1.6 Business Logic Overview

Logic	Where	What
Shortest Path	RouteService.cs	Dijkstra + PriorityQueue
Graph Building	GraphBuilder.cs	Adjacency dictionary, 30-min cache
Pricing	PricingService.cs	Tiered: 1-9→8EGP, 10-16→10EGP, 17+→15EGP
Travel Time	TravelTimeService.cs	stationCount × 2 minutes
Transfer Detection	TransferDetectionService.cs	Counts LineId changes

شرح المشروع بالمصري

بص يا معلم، المشروع ده اسمه Cairo Metro Route Planner. عبارة عن تطبيق ويب بيساعد الناس اللي راكبة مترو الأنفاق في القاهرة إنهم يعرفوا أحسن طريق يركبوه من محطة لمحطة.

ال Application معمول ب ASP.NET Core MVC، وده نظام **Metro**. 1. **三层 (3-Tier)** — ده طبقة ال UI، فيها ال Controllers وال **Metro.Core** Views — ده طبقة ال Business Logic، فيها ال Entities وال **Metro.Data** Services — ده طبقة ال Data، فيها ال DbContext وال Repositories

ال Database اسمها MetroDb وبتشتغل على SQL Server. فيها 4 جداول (Tables): **Lines** — بتخزن خطوط المترو (خط 1 أحمر، خط 2 أزرق، خط 3 أخضر) - **Stations** — بتخزن 89 محطة بإسمائها وإحداثياتها - **StationConnections** — بتخزن الوصلات بين المحطات (ال Graph edges) - **PricingRules** — بتخزن أسعار التذاكر حسب عدد المحطات

المستخدم بيعمل إيه؟ يدخل على الموقع، يختار محطة الانطلاق ومحطة الوصول من dropdowns، ويضغط Find Route. النظام يحسب: 1. أقصر طريق بين المحطتين (باستخدام Dijkstra algorithm) 2. سعر التذكرة (حسب عدد المحطات) 3. وقت الرحلة التقريبي (دقيقتين لكل محطة) 4. عدد مرات تغيير الخط (Transfer) كمان فيه خريطة Leaflet بتظهر المحطات، وميزة GPS اللي بتجيب أقرب محطة لموقع المستخدم.

لو الدكتور سألك

س: إيه هو المشروع بتاعك؟ ج: "يا دكتور، المشروع عبارة عن Cairo Metro Route Planner. تطبيق وب يساعد ركاب مترو الأنفاق في القاهرة إنهم يعرفوا أقصر طريق بين أي محطتين. بيحسب عدد المحطات وسعر التذكرة ووقت الرحلة وعدد مرات تغيير الخط."

س: Architecture بتاعة المشروع إيه؟ ج: "3-Metro Architecture. Tier ال Metro.Core، ال UI ال Business Logic، ال Metro.Data ال Data Access. وكمان استخدمنا MVC Pattern و Repository Pattern و Dependency Injection."

س: طبقة Metro.Core بتعمل إيه بالضبط؟ ج: "فيها ال Entities بتاعة المشروع زي Station و Line و StationConnection و PricingRule. وفيه ال Interfaces بتاعة ال Repositories وال Services. وفيه ال Services نفسها زي MetroService اللي بيعمل Orchestration وال RouteService اللي بيعمل Dijkstra وال PricingService اللي بيحسب السعر."

2. MVC ARCHITECTURE

Technical Content

2.1 Controllers

Controller	File	Purpose
RoutesController	Metro/Controllers/RoutesController.cs	Main route search — uses ViewModels + MetroService
HomeController	Metro/Controllers/HomeController.cs	Landing page — uses ViewBag (older approach)

2.2 Models (Entities)

Entity	File	Purpose
Station	Metro.Core/Entities/Station.cs	Metro station with name, coords, line
Line	Metro.Core/Entities/Line.cs	Metro line with name, color
StationConnection	Metro.Core/Entities/StationConnection.cs	Connection between two stations
PricingRule	Metro.Core/Entities/PricingRule.cs	Fare pricing tier

2.3 ViewModels

ViewModel	File	Properties
RouteSearchViewModel	Metro/ViewModels/RouteSearchViewModel.cs	FromStationId, ToStationId, Stations, Result, ErrorMessage
StationOptionViewModel	Metro/ViewModels/StationOptionViewModel.cs	Id, Name, LineName, Latitude, Longitude, DisplayName
ErrorViewModel	Metro/Models/ErrorViewModel.cs	RequestId, ShowRequestId

2.4 Views

View	Controller Served	Model Received
Views/Routes/Index.cshtml	RoutesController	RouteSearchViewModel
Views/Home/Index.cshtml	HomeController	none (ViewBag)
Views/Shared/_Layout.cshtml	All	—
Views/Shared/Error.cshtml	HomeController	ErrorViewModel
Views/_ViewStart.cshtml	All	—
Views/_ViewImports.cshtml	All	—

شرح MVC Architecture بالمصري

خلينا نفهم ال MVC Pattern الأول:

- **Model** — ده بيمثل البيانات. في مشروعنا، ال Models هي Station, Line, StationConnection, PricingRule. دول موجودين في Metro.Core/Entities.
- **View** — ده الواجهة اللي المستخدم بيشتورها. الملفات .cshtml في مجلد Views.
- **Controller** — ده الوسيط اللي بيستقبل ال Request من المستخدم، بيشتغل على ال Model، وبيختار ال View المناسبة.

المشروع فيه Controllerين:

- RoutesController**: - ده ال Controller الأساسي في المشروع - بيستخدم RouteSearchViewModel (وده أحسن من ViewBag) - عنده `[ValidateAntiForgeryToken]` — يعني محمي من CSRF - بيستخدم MetroService اللي بيعمل Orchestrate لكل حاجة
- HomeController**: - ده أقدم شوية - بيستخدم ViewBag لنقل البيانات — وده أضعف لأن ال ViewBag مش strongly-typed - مفيهوش `[ValidateAntiForgeryToken]` — دي مشكلة أمان - لو عايز تعرف الفرق: RoutesController أحسن بكثير
- ViewModels**: - RouteSearchViewModel ال **ViewModels**: ده بيتكون من FromStationId و ToStationId (اللي المستخدم بيختارهم)، Stations (لائحة المحطات لل dropdown)، Result (نتيجة الرحلة)، ErrorMessage (رسالة الخطأ) - StationOptionViewModel: ده بيمثل محطة واحدة في ال dropdown، وليه خاصية DisplayName اللي بتظهر "اسم المحطة (اسم الخط)"

لو الدكتور سألك

س: ليه في Controllerين؟ مش المفروض واحد؟ ج: "ده فعلاً نقطة ضعف في المشروع. RoutesController هو الأحدث والأحسن، و HomeController ده قديم. المفروض ندمجهم في واحد ونستخدم ال ViewModel pattern."

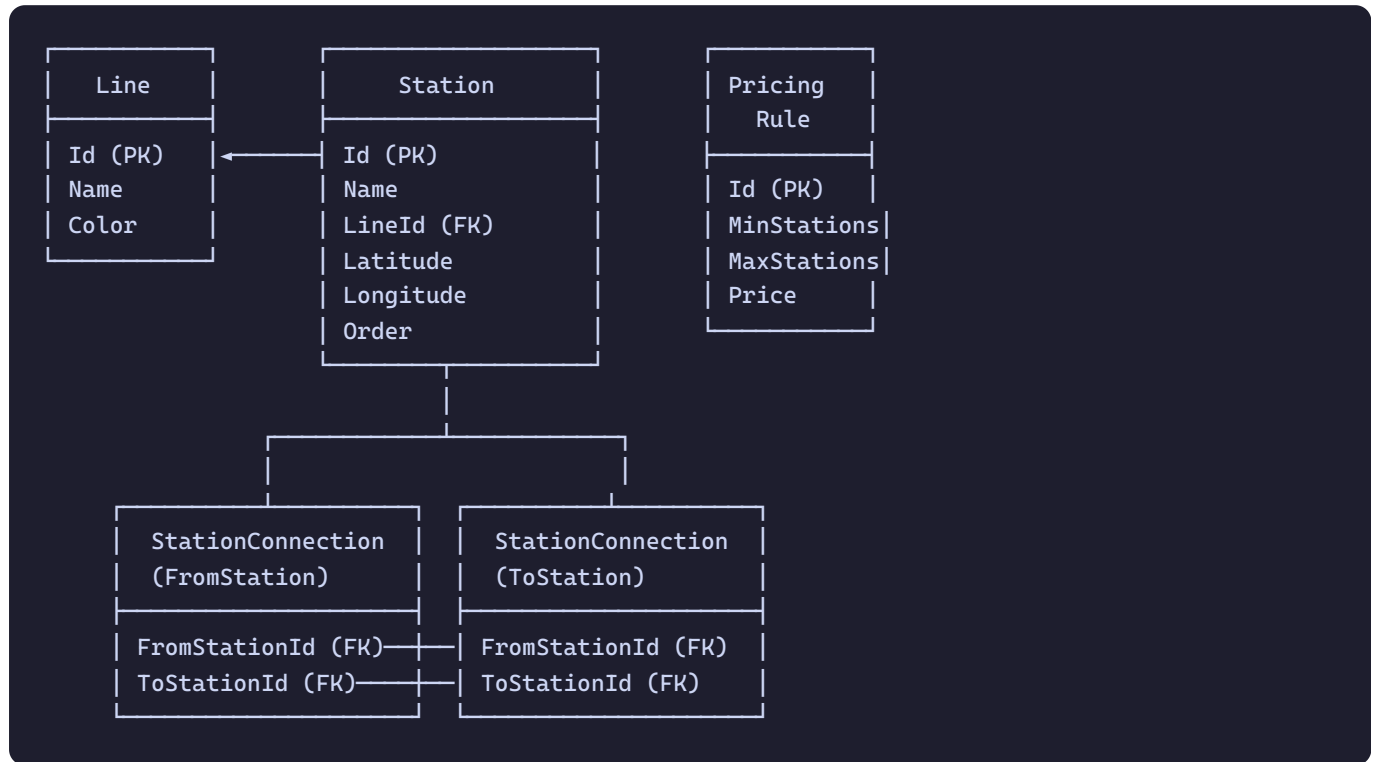
س: إيه الفرق بين RoutesController و HomeController؟ ج: "RoutesController يستخدم ViewModels و MetroService و HomeController يستخدم Anti-forgery protection. ViewBag ويستخدم مع ال Services مباشرة ومفيهوش Anti-forgery".

س: إيه أحسن حاجة في RoutesController؟ ج: "إنه يستخدم RouteSearchViewModel — ده -strongly typed، يعني أي غلطة في اسم ال Property تظهر وقت compilation مش وقت التشغيل. ويستخدم MetroService اللي بيعزل Business Logic عن ال Controller".

3. DATABASE ANALYSIS

Technical Content

3.1 Entity Relationship Diagram



3.2 Tables

Lines: Id (PK), Name, Color **Stations:** Id (PK), Name, LineId (FK→Lines), Latitude, Longitude, Order
StationConnections: Id (PK), FromStationId (FK→Stations), ToStationId (FK→Stations) **PricingRules:** Id (PK), MinStations, MaxStations, Price

3.3 Transfer Stations (5 Pairs)

- Sadat: Line1(Id=19) ↔ Line2(Id=46)
- Shohadaa: Line1(Id=22) ↔ Line2(Id=43)
- Attaba: Line2(Id=44) ↔ Line3(Id=74)
- Nasser: Line1(Id=20) ↔ Line3(Id=75)
- Cairo University: Line2(Id=50) ↔ Line3(Id=89)

3.4 Relationships

Relationship	Type	Why
Line → Station	One-to-Many	One line has many stations
Station → StationConnection (From)	One-to-Many	One station has many outgoing connections
Station → StationConnection (To)	One-to-Many	One station has many incoming connections

شرح Database بالمصري

تعالى نشوف ال Database بتاعتنا عليها إيه:

عندنا 4 Tables في SQL Server:

1. Lines Table: ده بيسجل ال 3 خطوط المترو. كل Line عنده Id و Color (Red, Blue, و Name (Line 1, Line 2, Line 3) و Green).

2. Stations Table: فيه 89 محطة. كل Station عنده Id و Name (زي Helwan, Sadat, Nasser) و LineId (بيشاو على الخط اللي المحطة عليه) و Latitude/Longitude (عشان الخريطة) و Order (ترتيب المحطة على الخط).

3. StationConnections Table: ده أهم جدول في ال Graph. كل Connection عنده FromStationId (المحطة اللي رايح منها) و ToStationId (المحطة اللي رايح ليها). يعني ييمثل خط في المترو بين محطتين.

4. PricingRules Table: فيه 3 قواعد تسعير: - من 1 ل 9 محطات: 8 جنيه - من 10 ل 16 محطة: 10 جنيه - من 17 محطة فأكثر: 15 جنيه

محطات التحويل (Transfer Stations): فيه 5 محطات بتتقاطع فيها الخطوط. يعني تقدر تنزل من خط وتركب خط تاني: - **سادات:** خط 1 (رقم 19) ↔ خط 2 (رقم 46) - **شهداء:** خط 1 (رقم 22) ↔ خط 2 (رقم 43) - **عتبة:** خط 2 (رقم 44) ↔ خط 3 (رقم 74) - **ناصر:** خط 1 (رقم 20) ↔ خط 3 (رقم 75) - **جامعة القاهرة:** خط 2 (رقم 50) ↔ خط 3 (رقم 89)

لو الدكتور سألك

س: إيه العلاقة بين Station و Line؟ ج: "Many-to-One". كل محطة بتنتمي لخط واحد (= Station.LineId). والخط الواحد عنده كتير محطات (Line.Stations).

س: ليه StationConnection عنده Foreign Key (FromStationId و ToStationId)؟ ج: "عشان ال Directional Connection بيكون — من محطة لمحطة. محطة ممكن يكون ليها وصلات خارجة (FromConnections) ووصلات داخلية (ToConnections). وده بيساعد في بناء ال Graph لـ Dijkstra algorithm".

س: إيه ال DeleteBehavior اللي استخدمته وليه؟ ج: "استخدمت `DeleteBehavior.Restrict` . يعني لو حاولت تحذف Line أو Station، ال Database مش هتسمح لو في حاجة بتشاور عليها. بتحمي ال Data من الحذف الغلط."

4. DBCONTEXT ANALYSIS

Technical Content

4.1 MetroDbContext (`Metro.Data/MetroDbContext.cs`)

```
public class MetroDbContext : DbContext
{
    public DbSet<Station> Stations { get; set; }
    public DbSet<Line> Lines { get; set; }
    public DbSet<StationConnection> StationConnections { get; set; }
    public DbSet<PricingRule> PricingRules { get; set; }
}
```

4.2 OnModelCreating

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(MetroDbContext).Assembly);
    base.OnModelCreating(modelBuilder);
}
```

Uses `ApplyConfigurationsFromAssembly` to auto-discover all `IEntityTypeConfiguration<T>` classes.

4.3 Fluent API Configurations

StationConfiguration: HasKey, ValueGeneratedNever, CompositeIndex(LineId, Order), HasOne/WithMany to Line, DeleteBehavior.Restrict **LineConfiguration:** HasKey, ValueGeneratedNever, Required MaxLength(100) Name, MaxLength(50) Color **StationConnectionConfiguration:** HasKey, ValueGeneratedNever, Indexes on From/ToStationId, Two HasOne/WithMany to Station, Restrict **PricingRuleConfiguration:** HasKey, ValueGeneratedNever

شرح DbContext بالمصري

ال DbContext ده هو الجسر بين الكود بتاعنا و SQL Server.

إحنا عندنا `MetroDbContext` بيرث من `DbContext`. فيه 4 `DbSet` — واحد لكل جدول في ال Database. الميثود `OnModelCreating` بتستعمل `ApplyConfigurationsFromAssembly` عشان تلاقي كل ال Configurations ال Fluent API اللي كتبناها.

خلي بالك من حاجة مهمة: كل ال Entities عندنا استعملت `.ValueGeneratedNever()`. على ال Id. يعني إحنا اللي بنحدد ال Id بنفسنا مش ال EF Core. ليه؟ عشان ال Seed Data جاي من ملفات JSON وال Ids فيها محددة سلفاً.

كمان استخدمنا `DeleteBehavior.Restrict` على كل ال Foreign Keys — يمنع الحذف المتسلسل (Cascade Delete).

لو الدكتور سألک

س: إيه وظيفة `ApplyConfigurationsFromAssembly` ؟ ج: "بتدور على كل Class 实现了 في نفس ال Assembly بتاع MetroDbContext وتطبقهم تلقائيًا."

س: ليه استخدمت `ValueGeneratedNever()` ؟ ج: "عشان ال Seed Data جاي من JSON وال Ids فيها محددين مسبقًا. لو سيبناه EF Core يولد Ids تلقائيًا، هتضرب مع ال Ids الجاهزة."

س: إيه الفرق بين `Cascade` و `Restrict` في `Delete` ؟ ج: "Cascade يسمح كل الحاجات اللي مرتبطة بال Record. Restrict يمنع المسح خالص لو في حاجة بتشاور عليه."

5. CONTROLLER DEFENSE GUIDE

5.1 RoutesController

File: `Metro/Controllers/RoutesController.cs`

Dependencies

```
private readonly IStationRepository _stationRepository;  
private readonly IMetroService _metroService;
```

Action: Index (GET)

```
[HttpGet]  
public async Task<IActionResult> Index()  
{  
    var viewModel = new RouteSearchViewModel  
    {  
        Stations = await LoadStationsAsync()  
    };  
    return View(viewModel);  
}
```

Creates ViewModel with all stations for dropdowns, returns the view.

Action: Index (POST)

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Index(RouteSearchViewModel model)
{
    if (model.FromStationId is null || model.ToStationId is null)
    {
        model.ErrorMessage = "Please select both stations.";
        model.Stations = await LoadStationsAsync();
        return View(model);
    }
    if (model.FromStationId == model.ToStationId)
    {
        model.ErrorMessage = "Stations cannot be the same.";
        model.Stations = await LoadStationsAsync();
        return View(model);
    }
    try
    {
        RouteResultDto result = await _metroService.GetRouteAsync(
            model.FromStationId.Value, model.ToStationId.Value);
        model.Result = result;
        model.Stations = await LoadStationsAsync();
    }
    catch (Exception ex)
    {
        model.ErrorMessage = $"Could not calculate route: {ex.Message}";
        model.Stations = await LoadStationsAsync();
    }
    return View(model);
}
```

Private Helper: LoadStationsAsync

```
private async Task<List<StationOptionViewModel>> LoadStationsAsync()
{
    var stations = await _stationRepository.GetAllAsync();
    return stations
        .Select(s => new StationOptionViewModel
        {
            Id = s.Id, Name = s.Name,
            LineName = s.Line?.Name ?? string.Empty,
            Latitude = s.Latitude, Longitude = s.Longitude
        })
        .OrderBy(s => s.Name).ToList();
}
```


شرح RoutesController بالمصري

خليني أشرحك RoutesController بالتفصيل:

- ال Controller ده عليه مسؤولية Route Search — البحث عن الطريق. عنده Dependency على: 1. `IStationRepository` — عشان يجيب المحطات من ال Database 2. `IMetroService` — عشان يحسب الرحلة
- ال **GET Index**: لما المستخدم يدخل على صفحة `/Routes/Index` لأول مرة، بنعمل ViewModel جديد. بنستدعي `LoadStationsAsync()` اللي بتجيب كل المحطات من ال Repository وترتبهم أبجديًا وترجعهم كـ `StationOptionViewModel`. بنيعت ال ViewModel لل View اللي بتظهر ال Form.
- ال **POST Index**: لما المستخدم يضغط 1. Find Route. بنستقبل ال `RouteSearchViewModel` (ال Model Binding يجيب البيانات من ال Form) 2. بنعمل Validation يدوي: - نتأكد إن `FromStationId` و `ToStationId` مش null - نتأكد إنهم مختلفين 3. لو في غلط، بنضبط `ErrorMessage` ونرجع ال View تاني 4. لو صح، بنستدعي `_metroService.GetRouteAsync()` — ده بيحسب الرحلة كلها 5. لو حصل `Exception`، بنضبط `ErrorMessage` برضه 6. في كل الحالات بنعيد تحميل `Stations` عشان ال Dropdowns
- `[ValidateAntiForgeryToken]`: ده بيحمي من CSRF attacks — لو حد حاول بيعت Request من موقع تاني، مش هينفع.
- ال **LoadStationsAsync**: `GetAllAsync()` بتجيب كل المحطات بـ Include لل Line (عشان يجيب اسم الخط). بنستخدم `?.` عشان لو ال Line كان (defensive programming) null. وبنستخدم `?.string.Empty` عشان لو كان null، نستخدم نص فاضي. وبعدين بنرتب بـ `OrderBy(s => s.Name)` أبجديًا.

أسئلة الدكتور المتوقعة على RoutesController

س: ليه بتستعمل `LoadStationsAsync` مرتين في ال POST؟ ج: "عشان ال View محتاجة `Stations` عشان تظهر ال Dropdowns. سواء نجح ال Route Calculation أو لا، لازم أرجع `Stations` تاني."

س: إيه اللي هيحصل لو شلت `[ValidateAntiForgeryToken]`؟ ج: "ال CSRF Attack هتكون ممكنة. أي موقع تاني يقدر بيعت POST Request للمستخدم وهو مش عارف."

س: ليه بترجع `View(model)` مش `Redirect`؟ ج: "عشان نفضل على نفس الصفحة ونشوف النتيجة. ال POST-Redirect-GET Pattern هنا مش ضروري لأننا بنعرض النتيجة مش بنعدل بيانات."

س: إيه الفرق بين `is null` و `==null`؟ ج: "`is null` هي نفس `==null` لكن `is null` مش بتعتمد على ال `overloaded == operator`. الفرق النظري بس — عمليًا واحد."

5.2 HomeController

File: `Metro/Controllers/HomeController.cs`

Dependencies

```
private readonly IStationRepository _stationRepository;  
private readonly IRouteService _routeService;  
private readonly IPricingRuleRepository _pricingRuleRepository;
```

POST Action

```
[HttpPost]  
public async Task<IActionResult> Index(int fromStationId, int toStationId)  
{  
    // No [ValidateAntiForgeryToken] - SECURITY ISSUE  
    var stations = await _stationRepository.GetAllAsync();  
    ViewBag.Stations = stations;  
    if (fromStationId == toStationId) { ViewBag.Error = "..."; return View(); }  
    var pathIds = await _routeService.GetShortestPathAsync(fromStationId, toStationId);  
    var routeStations = pathIds.Select(id => stations.FirstOrDefault(s => s.Id == id)) ...;  
    var price = _pricingRuleRepository.GetAllAsync() ...  
        .FirstOrDefault(rule => rule.IsMatch(stationCount))?.Price ?? 0;  
    // ... sets ViewBag properties  
    return View();  
}
```

شرح HomeController بالمصري

ال HomeController ده أقدم شوية من RoutesController ومش بنفس المستوى.

المشاكل اللي فيه: 1. مفيش `[ValidateAntiForgeryToken]` — ده ثغرة أمنية 2. بيستخدم `ViewBag` — مش strongly-typed, يعني لو غلطت في اسم Property مفيش Compile Error 3. ال **Business Logic** متفرقة — مش centralized زي MetroService

طب إيه ال **positive**؟ ال POST بيستخدم `IRouteService` و `IPricingRuleRepository` مباشرة. لكن RoutesController أحسن بكتير.

أسئلة الدكتور المتوقعة على HomeController

س: ليه ال POST ال HomeController مفيهوش [ValidateAntiForgeryToken] ؟ ج: "ده ثغرة. المفروض يتضاف. RoutesController عنده الحماية دي."

س: إيه المشكلة في ViewBag ؟ ج: "ViewBag بيستخدم String ك Key. لو كتبت ViewBag.Stationssss مش هيعمل Error — هيرجع null بس. لكن لو استخدمت ViewModel، أي غلطة في اسم ال Property تظهر في Compile Time."

س: الفرق بين 0 ?? Price. إيه؟ ج: "Price." — لو ال PricingRule كان null، يرجع null من غير NullReferenceException. 0 ?? — لو النتيجة null، استخدم 0 بدالها."

6. PROGRAM.CS DEFENSE

Technical Content

File: `Metro/Program.cs`

```
var builder = WebApplication.CreateBuilder(args);

// Services
builder.Services.AddControllersWithViews();
builder.Services.AddDbContextPool<MetroDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// Repositories
builder.Services.AddScoped<IStationRepository, StationRepository>();
builder.Services.AddScoped<ILineRepository, LineRepository>();
// ... other repositories

// Cache
builder.Services.AddMemoryCache();

// Services
builder.Services.AddScoped<IGraphBuilder, GraphBuilder>();
builder.Services.AddScoped<IRouteService, RouteService>();
// ... other services

var app = builder.Build();

// Middleware Pipeline
if (!app.Environment.IsDevelopment()) {
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();
app.MapControllerRoute(name: "default", pattern: "{controller=Home}/{action=Index}/{id?}");

// Database Seeding
using (var scope = app.Services.CreateScope()) {
    var context = scope.ServiceProvider.GetRequiredService<MetroDbContext>();
    var logger = scope.ServiceProvider.GetRequiredService<ILogger<Program>>();
    await MetroDataSeeder.SeedAsync(context, logger);
}
app.Run();
```

شرح Program.cs بالمصري

الـ Program.cs هو نقطة بداية الـ Application. خلينا نمشي عليه سطر سطر:

1. `builder.Services.AddControllersWithViews()` — يسجل MVC Services. من غيرها، أي Controller يطلع 404.
2. `AddDbContextPool<MetroDbContext>` — يسجل الـ DbContext مع Connection Pooling. الفرق بينها وبين `AddDbContext` : Pooling بتعيد استخدام الـ Context instances وبتحسن Performance.
3. `AddScoped` للـ Repositories والـ Services — يعني كل Request جديد ياخذ instance جديدة. مناسب لـ Web Apps.
4. `AddMemoryCache()` — عشان نخزن الـ Graph ونستخدمه تاني بدل ما نبنيه من الأول كل شوية.
5. الـ Middleware Pipeline ترتيبها مهم:
6. `UseExceptionHandler` — للـ Production، لو حصل Error يوديك على `/Home/Error`
7. `UseHttpsRedirection` — يحول HTTP لـ HTTPS
8. `UseStaticFiles` — يخدم ملفات CSS/JS من `wwwroot`
9. `UseRouting` — يشوف الـ Request ينفع لأي Route
10. `UseAuthorization` — جهاز للأمان (مستخدمش حالياً)
11. `MapControllerRoute` — يحدد الـ Routes الافتراضية
12. الـ Database Seeding — بيتعمل كل ما الـ App يشتغل. بيحيب بيانات JSON ويضيفها للـ Database، بس بيدور على Ids موجودة مسبقاً عشان يمنع التكرار.

لو الدكتور سألك

س: ليه استخدمت `AddDbContextPool` مش `AddDbContext` ؟ ج: "عشان DbContext. Performance. Pooling بيسمح بإعادة استخدام instances بدل ما نعمل new instance مع كل Request."

س: إيه ترتيب Middlewares وليه مهم؟ ج: "الترتيب مهم جداً. HTTPS Redirection لازم تكون قبل Static Files. Routing لازم يكون قبل Authorization. لو غيرت الترتيب، ممكن الحاجات تشتغل غلط."

س: ليه Seeding بتتعمل جوا (`using (var scope = ...)` ؟ ج: "عشان MetroDbContext مسجل كـ Scoped Service. الـ Program.cs نفسه مش جوا Request، فلزام نعمل Scope جديد عشان نقدر نجيب الـ Service."

7. DEPENDENCY INJECTION

Technical Content

Interface	Implementation	Lifetime	Used By
IStationRepository	StationRepository	Scoped	HomeController, RoutesController, GraphBuilder, MetroService
ILineRepository	LineRepository	Scoped	(unused — dead code)
IStationConnectionRepository	StationConnectionRepository	Scoped	GraphBuilder
IPricingRuleRepository	PricingRuleRepository	Scoped	HomeController, MetroService
IGraphBuilder	GraphBuilder	Scoped	RouteService
IRouteService	RouteService	Scoped	HomeController, MetroService
IMetroService	MetroService	Scoped	RoutesController
IPricingService	PricingService	Scoped	MetroService
ITravelTimeService	TravelTimeService	Scoped	MetroService
ITransferDetectionService	TransferDetectionService	Scoped	MetroService

DI Chain (6 levels deep)

```
RoutesController → IMetroService → IRouteService → IGraphBuilder
    → IStationRepository → MetroDbContext
    → IStationConnectionRepository → MetroDbContext
    → IMemoryCache
    → ITravelTimeService
    → IPricingService
    → ITransferDetectionService
    → IStationRepository
    → IPricingRuleRepository
```

شرح Dependency Injection بالمصري

الـ DI ببساطة: بدل ما الـ Controller يعمل `new StationRepository()` ، هو بيقول "أنا عايز `IStationRepository`" والـ Container بيديهوله.

ليه ده أحسن؟ 1. **Loose Coupling** — ال Controller ميعرفش إزاي ال Repository بيتعمل، بس بيعرف إنه بيطبق ال Interface
 2. **Testability** — تقدر تعمل Mock لل Interface وتختبر ال Controller بمعزل 3. **Lifecycle Management** — ال Container هو اللي بيدير ال Instance ازاي تتعمل وتموت
 كل ال Services عندنا مسجلة بـ **AddScoped** — يعني واحدة لكل Request. لو استخدمنا Singleton، هيحصل Sharing بين ال Requests وده خطر.

ال Chain وصلت لـ 6 Repository → GraphBuilder → RouteService → MetroService → RoutesController
Levels: DbContext

ده طول طبيعي في ال Enterprise Applications وده علامة على Separation of Concerns.

لو الدكتور سألَك

س: إيه الفرق بين AddSingleton, AddScoped, AddTransient؟ ج: "instance — Singleton واحدة لل App كله. instance — Scoped لكل instance — Transient Request. جديدة كل ما حد يطلبها."

س: ليه استخدمت AddScoped لكل Services؟ ج: "عشان كل Request ياخد Instance جديدة. كده ال DbContext مش هيتشارك بين Requests ونتجنب مشاكل ال Thread Safety."

س: إيه ال Chain اللي بيحصل لما ال RoutesController يتطلب؟ ج: "ال DI Container بيحل MetroService اللي محتاج 6 Dependencies. وهما برضه ليهم Dependencies. ال Container بيحل الشجرة كلها مرة واحدة."

س: فيه Service مسجل ومش مستخدم؟ ج: "أيوه — ILineRepository. مسجل في Program.cs لكن مفيش Controller أو Service بيستخدمه. ده Dead Code."

8. ENTITY FRAMEWORK CORE

Technical Content

Key EF Core Features Used

Feature	Where	Why
<code>.Include(s => s.Line)</code>	StationRepository	Eager loading — JOIN with Line table
<code>.AsNoTracking()</code>	Graph builder queries	Read-only data, better performance
<code>.Select()</code> projection	Graph node/edge queries	Only needed columns
<code>ValueGeneratedNever()</code>	All entity configs	IDs from JSON seed data
<code>AddRangeAsync()</code>	MetroDataSeeder	Batch insert optimization
<code>FirstOrDefaultAsync()</code>	Repository queries	Returns null if not found
<code>ToListAsync()</code>	All queries	Materializes results

Include generates SQL:

```
SELECT s.*, l.* FROM Stations s LEFT JOIN Lines l ON s.LineId = l.Id
```

AsNoTracking with projection generates:

```
SELECT Id, Name, LineId FROM Stations
```

شرح EF Core بالمصري

Entity Framework Core هو ORM (Object Relational Mapper) — يحوّل الـ C# Objects لـ SQL Queries والعكس.

الـ Features اللي استخدمناها في المشروع:

1. `.Include(s => s.Line)` — بيعمل JOIN مع جدول Lines عشان نجيب اسم الخط مع المحطة. من غيرها، لو حاولت توصل لـ `station.Line` هيرجع null.
2. `.AsNoTracking()` — بيقول لـ EF "مش هتغير في البيانات دي، متتعيش نفسك في Tracking". أسرع ويبستهلك ذاكرة أقل.

3. **.Select()** **Projection** — بدل ما نجيب كل Columns المحطة، بنجيب بس اللي محتاجينه: Id, Name, LineId. ال SQL الناتج: `SELECT Id, Name, LineId FROM Stations`.
4. **ValueGeneratedNever()** — أهم حاجة: بنقول ل EF "ماتولدش ال Id بنفسك، إحنا اللي هنعده." عشان ال Seed Data جاهز من JSON وليه Ids محددة.
5. **AddRangeAsync()** — بنضيف كل الحاجات مرة واحدة بدل واحدة واحدة. بيحول ل INSERT واحد بجمل كتير.
6. **FirstOrDefaultAsync()** — بترجع أول مطابقة أو null لو مفيش. أحسن من **First()** اللي بيعمل Exception لو مفيش.

لو الدكتور سألك

س: إيه الفرق بين First و FirstOrDefault ؟ ج: **First** " بيعمل Exception لو مفيش نتيجة. **FirstOrDefault** بترجع null. في المشروع بنستخدم **FirstOrDefault** عشان نعمل null-check ونحدد ال Error message بنفسنا."

س: إيه ال SQL اللي Include(s => s.Line) بيولدها؟ ج: `SELECT s.*, l.* FROM Stations s LEFT JOIN Lines l ON s.LineId = l.Id`

س: ليه استخدمت AsNoTracking في ال Graph Queries ؟ ج: "عشان البيانات دي للقراءة بس (Read-only). Tracking بيستهلك Memory ووقت من غير فائدة. **AsNoTracking** بيسرع ال Query ويقلل Memory Usage."

س: إيه اللي هيجصل لو شلت AsNoTracking ؟ ج: "ال Change Tracker هيتابع ال Objects اللي مش هيجتاج يتغيروا. ال Performance هتقل شوية وال Memory هتزيد."

9. LINQ ANALYSIS

Technical Content

All LINQ Queries

#	Query	Location	SQL Equivalent
1	<code>.Include(s => s.Line).FirstOrDefaultAsync(s => s.Id == id)</code>	StationRepository	<code>SELECT TOP 1 ... FROM Stations JOIN Lines WHERE Id = @id</code>
2	<code>.Include(s => s.Line).ToListAsync()</code>	StationRepository	<code>SELECT * FROM Stations JOIN Lines</code>
3	<code>.Where(s => s.LineId == lineId).OrderBy(s => s.Order).ToListAsync()</code>	StationRepository	<code>SELECT * FROM Stations WHERE LineId = @id ORDER BY Order</code>
4	<code>.AsNoTracking().Select(s => new StationGraphNode { ... }).ToListAsync()</code>	StationRepository	<code>SELECT Id, Name, LineId FROM Stations</code>
5	<code>.AsNoTracking().Select(c => new StationConnectionGraphNode { ... }).ToListAsync()</code>	StationConnectionRepo	<code>SELECT FromStationId, ToStationId FROM StationConnections</code>
6	<code>.Where(sc => sc.FromStationId == id sc.ToStationId == id).ToListAsync()</code>	StationConnectionRepo	<code>SELECT * FROM StationConnections WHERE FromStationId = @id OR ToStationId = @id</code>
7	<code>.OrderBy(r => r.MinStations).ToListAsync()</code>	PricingRuleRepo	<code>SELECT * FROM PricingRules ORDER BY MinStations</code>
8	<code>.Include(l => l.Stations).FirstOrDefaultAsync(l => l.Id == id)</code>	LineRepository	<code>SELECT TOP 1 ... FROM Lines JOIN Stations WHERE Id = @id</code>

شرح LINQ Queries بالمصري

طيب خلينا نستعرض كل LINQ Query في المشروع ونفهمها:

Query 1 & 2: بنجيب كل المحطات أو محطة واحدة بـ Include لـ Line. لازم Include عشان نجيب اسم الخط مع المحطة. لو معرفش Include، `station.Line` هيرجع null.

- Query 3:** بنجيب محطات خط معين وبنرتبهم حسب ال Order. ده بيستخدم في عرض محطات خط معين بالترتيب. ال Composite Index على (LineId, Order) بيخلي ال Query دي سريعة.
- Query 4 & 5:** بنجيب Ids المحطات والوصلات عشان بنبي ال Graph. بنستخدم AsNoTracking عشان البيانات للقراءة بس. وبنستخدم Select عشان نجيب Ids بس مش كل البيانات.
- Query 6:** بنجيب كل الوصلات اللي ليها علاقة بمحطة معينة — سواء هي البداية أو النهاية. بيستخدم في ال Search.
- Query 7:** بنجيب Rules التسعير مرتبة حسب MinStations عشان نضمن إن أول Rule يطابق هو ال Tier الصحيح.
- Query 8:** بنجيب خط معين وكل محطاته. مستخدمش في أي Controller حالياً (Dead Code).

لو الدكتور سألك

س: في Query 4، ليه استخدمت AsNoTracking مع Select ؟ ج: "عشان بنجيب Ids بس مش Objects كاملة. ال Change Tracker مش محتاج يتابع حاجة. ال Performance أحسن وال Memory أقل."

س: إيه الفرق بين Select بعد Include. وقبله؟ ج: "لو استخدمت Select. بعد Include. ، ال Include بيتلغى. لأن ال Select بيخلق نوع جديد مش Station. ف ال Include ملوش لازمة مع ال Select."

س: ليه Query 3 بتستخدم OrderBy لكن Query 2 لأ؟ ج: "لأن في Query 3 بنجيب محطات خط واحد ونعرضهم بالترتيب (Order مهم). في Query 2 بنجيب كل المحطات لل Dropdown — الترتيب أبجدي حسب الاسم مش حسب Order."

10. VALIDATION & DATA ANNOTATIONS

Technical Content

Current Validation State

The project uses **manual validation** in controllers — no data annotations.

RoutesController Validation:

```
if (model.FromStationId is null || model.ToStationId is null)
{
    model.ErrorMessage = "Please select both a departure station and a destination station.";
    return View(model);
}
if (model.FromStationId == model.ToStationId)
{
    model.ErrorMessage = "Departure and destination stations cannot be the same.";
    return View(model);
}
```

Client-side validation (JavaScript):

```
document.getElementById('route-search-form').addEventListener('submit', function (e) {
    if (from === to) {
        e.preventDefault();
        showClientError('Departure and destination stations cannot be the same.');
```

Missing: No `[Required]` attributes, no `ModelState.IsValid` check, no data annotations on ViewModels.

شرح Validation بالمصري

ال Validation في المشروع معمول Manual في ال Controller — مش باستخدام Data Annotations.

RoutesController بيتأكد من حاجتين: 1. إن المستخدم اختار المحطتين (مش سايب حاجة فاضية) 2. إن المحطتين مختلفتين لو في غلط، بنحط رسالة في `model.ErrorMessage` ونرجع ال View ثاني عشان المستخدم يشوف الخطأ.

طب فين المشكلة؟ مفيش `[Required]` Attributes على ال ViewModel. يعني ال `ModelState.IsValid` دايماً true. لو استخدمنا Data Annotations، ال Validation هتشتغل Server-side و Client-side (من غير ما نكتب JavaScript).

ال Client-side Validation: ال JavaScript بتمنع Submit لو المحطتين نفس بعض — ده Client-side validation بس. يعني لو ال JavaScript مش شغال أو حد استخدم Postman، ال Server Validation (Manual) هو اللي هيمسك الغلط.

HTML5 Required: ال `<select>` عنده `required` attribute — ده برضه Client-side validation. لو المستخدم معطل JavaScript، ال Browser نفسه هيمنع ال Form Submission لو حاجة فاضية.

لو الدكتور سألك

س: ليه مش بتستخدم `ModelState.IsValid` ؟ ج: "عشان مفيش Data Annotations على ال `ViewModel`. استخدمت Manual Validation. لكن Data Annotations أحسن لأنها تشتغل Server و Client مع بعض."

س: إيه اللي هيجصل لو ضفت `[Required]` على `FromStationId` ؟ ج: "ال `ModelState.IsValid` هتكون False لو القيمة فاضية. وهتظهر رسالة خطأ أوتوماتيك. ومحتاج أغير ال Controller إنه يشيك على `ModelState` بدل ال `if-statement`"

س: هل ال Client-side Validation كافية؟ ج: "لا خالص. أي حد يقدر يعطل JavaScript أو بيعت Request من Postman. ال Server-side Validation هي الحماية الحقيقية."

11. SECURITY

Technical Content

Present Security Measures

Measure	Where	Status
Anti-Forgery Token	RoutesController POST	✓ With <code>[ValidateAntiForgeryToken]</code> + <code>@Html.AntiForgeryToken()</code>
HTTPS Redirection	Program.cs	✓
HSTS	Program.cs (non-Dev)	✓
Error Handler	Program.cs	✓
ResponseCache(NoStore)	Error action	✓

Missing Security Measures

Measure	Missing From	Risk
<code>[ValidateAntiForgeryToken]</code>	HomeController POST	CSRF vulnerability
Authentication	All	Not needed — public app
Rate Limiting	None	Low risk

Anti-Forgery Token Flow

1. Server generates unique token, embeds in form via `@Html.AntiForgeryToken()`
2. Same token set as cookie
3. On POST, server validates form token = cookie token
4. If mismatch → 400 Bad Request

شرح Security بالمصري

خلينا نكلم بصراحة عن ال Security في المشروع:

الحاجات اللي موجودة: - RoutesController عنده `[ValidateAntiForgeryToken]` — ده يحمي من هجمات CSRF. يعني لو موقع تاني حاول يبعث Request على Routes/Index، مش هينفع لأن ال Token مش هيتطابق. - `UseHttpsRedirection`

يحول HTTP لـ HTTPS - Error Handler — لو حصل خطأ، بيوديك على صفحة Error بدل ما يظهر Exception كامل

الحاجات اللي ناقصة: - HomeController POST مفيهوش `[ValidateAntiForgeryToken]` — ده ضعف - Authentication مفيهوش — بس ده عادي لأن الـ App عام ومافيش Users - مفيش Rate Limiting — بس برضه عادي للمشروع ده

إزاي Anti-Forgery Token بيشتغل: 1. الـ Server يعمل Token فريد ويحطه في الـ Form 2. نفس الـ Token يتحط في الـ Cookie 3. لما الـ Form يت Submit، الـ Server يتأكد إن الـ Token من الـ Form = الـ Token من الـ Cookie 4. لو مختلفين <= Bad Request 400

لو الدكتور سألك

س: إيه هي CSRF Attack؟ ج: "هجوم الـ Cross-Site Request Forgery. موقع ضار يعمل Form بيعت Request لموقعنا من غير المستخدم ما يعرف. الـ Anti-Forgery Token بيمنع ده."

س: ليه RoutesController عنده الحماية و HomeController لأ؟ ج: "ده oversight. المفروض HomeController يضيف `[ValidateAntiForgeryToken]` برضه. RoutesController مثال أحسن في الـ Security."

س: إيه اللي بيحصل لو الـ Token مش متطابق؟ ج: "الـ Server يرد بـ Bad Request 400. الـ Form مش هيتقدم."

س: هل Overposting (Mass Assignment) ممكنة؟ ج: "في RoutesController — لأ. عشان بيستخدم ViewModel. حتى لو المستخدم بعث extra fields، الـ Model Binding ماأخدش غير اللي في الـ ViewModel. في HomeController — لأ، عشان بيستخدم Primitive Parameters."

12. RAZOR & VIEWS

Technical Content

Tag Helpers Used

```
<form asp-controller="Routes" asp-action="Index" method="post">  
    @Html.AntiForgeryToken()
```

- `asp-controller` , `asp-action` generate correct URLs
- `@Html.AntiForgeryToken()` generates hidden CSRF token
- `asp-append-version="true"` on CSS files for cache busting

Layout Structure

```
_RenderBody() - child view content  
Footer: © 2026 - Metro  
_RenderSectionAsync("Scripts", required: false)
```

ViewStart / ViewImports

```
// _ViewStart: Layout = "_Layout"  
// _ViewImports: @using Metro, @using Metro.Models, @addTagHelper
```

Client-Side Libraries

Library	Purpose
Tom Select	Searchable dropdowns
Leaflet + OpenStreetMap	Interactive map
jQuery Validation	Client-side validation
Geolocation API	GPS nearest station

شرح Razor & Views بالمصري

ال Views في المشروع معمولة بـ (.cshtml) Razor وال Layout بيحتوي على:

1. `RenderBody()` — المكان اللي المحتوى الأساسي للصفحة بيظهر فيه

2. Footer — حقوق النشر

3. `RenderSectionAsync("Scripts")` — مكان الـ Scripts اللي الصفحات بتحتها

Tag Helpers - `asp-controller` و `asp-action` — بيحددوا الـ Controller و Action للـ Form. بدل ما تكتب URL يدوي، بنقول `asp-controller="Routes" asp-action="Index"` وهو يولد URL صح. - `@Html.AntiForgeryToken()` — يحط Hidden Input بيهدف الـ CSRF Token - `asp-append-version="true"` — يحط Hash Query String على الـ CSS عشان الـ Browser يعرف إن الملف اتغير ويحمله جديد.

Client-Side Libraries: - Tom Select الـ `<select>` العادي لـ Searchable Dropdown. مفيد جدًا لأن عندنا 89 محطة. **Leaflet**: خريطة تفاعلية من OpenStreetMap. بتظهر المحطات وموقع المستخدم. **jQuery Validation: Validation** - على الـ **Geolocation API** - Client. لـ جلب موقع المستخدم بالـ GPS.

إزاي الخريطة بتشتغل:

```
var map = L.map('map').setView([30.0444, 31.2357], 11); // Cairo coordinates
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(map);
```

بتظهر خريطة القاهرة على مستوى 11 Zoom. الـ Tiles من OpenStreetMap.

Tom Select Initialization

```
var fromSelect = new TomSelect('#FromStationId', {
  create: false,
  sortField: { field: "text", direction: "asc" }
});
```

بتخلي الـ Dropdown قابل للبحث وتظهر الخيارات مرتبة أبجديًا.

لو الدكتور سألك

س: إيه فائدة `asp-append-version` ؟ ج: "بيحط Hash Query String على الـ File URL. لو الملف اتغير، الـ Hash يتغير. الـ Browser يشوف URL جديد ويحمل الملف تاني بدل ما يستعمل Cache قديم."

س: إيه الفرق بين `@Html.ValidationSummary(false)` و `true` ؟ ج: "`false`" يعني عرض كل Errors الـ Model-Level والـ Property-Level. `true` يعني عرض Model-Level بس. إحنا استخدمنا `false` عشان نضمن كل الأخطاء تظهر."

س: ليه استخدمت CDN للـ Leaflet مش Local Files ؟ ج: "عشان أسهل وبيوفر Traffic على الـ Server. بس لو الإنترنت قطع، الخريطة مش هتشتغل. الـ Trade-off موجود."

س: إزاي الـ GPS بيشتغل في الموقع؟ ج: "المستخدم يضغط Use My Location. الـ Browser يطلب Permission. لو وافق، بـ `navigator.geolocation.getCurrentPosition()` نجيب إحداثيات المستخدم. بـ Haversine Formula نحسب أقرب محطة. ونختارها في الـ Dropdown أوتوماتيك."

13. PROJECT FLOW WALKTHROUGH

Complete Scenario: User Finds Route

Step 1: User opens `/Routes/Index` **Step 2:** `RoutesController.Index (GET)` — loads stations, returns View with empty form **Step 3:** Browser renders — two Tom Select dropdowns, Leaflet map, Find Route button **Step 4:** User selects Helwan → Nasser, clicks Find Route **Step 5:** Client JS validates stations are different, form submits POST `/Routes/Index` **Step 6:** Model binding creates `RouteSearchViewModel` with `FromStationId=1`, `ToStationId=20` **Step 7:** Controller validates → calls `_metroService.GetRouteAsync(1, 20)` **Step 8:** `MetroService` validates stations exist → calls `_routeService.GetShortestPathAsync(1, 20)` **Step 9:** `RouteService` calls `GraphBuilder.BuildGraphAsync()` — checks cache, miss, builds from DB **Step 10:** Dijkstra runs: processes stations from 1 to 20, finds path: `[1,2,3,...,20]` **Step 11:** Back in `MetroService` : resolves station names, calculates `travelTime=40min`, `transfers=0`, `price=15EGP` **Step 12:** Returns `RouteResultDto` to controller **Step 13:** Controller sets `model.Result = result`, reloads Stations, returns View **Step 14:** Razor renders: endpoint pills (Helwan → Nasser), stats (20 stations, 0 transfers, 15EGP, 40min), path list **Step 15:** JS scrolls result into view

شرح تدفق المشروع بالمصري

خلينا نمشي خطوة بخطوة مع المستخدم:

1. المستخدم يفتح الصفحة: يكتب في المتصفح `/Routes/Index`
2. ال Controller يستقبل الطلب: ال GET Action يعمل ViewModel جديد ويحمل كل المحطات من ال Database
3. الصفحة تظهر: المستخدم يشوف Form فيه Dropdowns للمحطات وخريطة
4. المستخدم يختار المحطات: اختار Helwan (رقم 1) و Nasser (رقم 20) على خط 1
5. ال Client Validation: JavaScript يتأكد إن المحطتين مختلفين
6. ال Form يت Submit: POST Request على نفس ال URL
7. ال Controller يستقبل البيانات: Model Binding يملأ RouteSearchViewModel
8. ال Validation: Controller يتأكد إن البيانات صح
9. ال MetroService بيشتغل: يتأكد إن المحطتين موجودين في Database
10. ال Dijkstra بيحسب الطريق: GraphBuilder يبني Graph و Dijkstra يلاقي أقصر طريق
11. النتيجة ترجع: RouteResultDto بـ 20 محطة، 0 تغيير خط، 40 دقيقة، 15 جنيه
12. ال Controller بيعت البيانات لل View: Result + Stations
13. ال View يعرض النتيجة: بطاقة النتيجة تظهر مع إحصائيات الرحلة
14. المستخدم يشوف الرحلة كلها: قائمة المحطات بالترتيب من Helwan لـ Nasser

14. احكيلي المشروع كأي بشرحه للدكتور

بص يا دكتور، خليني أشرحك المشروع من الأول:

المشروع اسمه **Cairo Metro Route Planner**. هو تطبيق وب يساعد ركاب مترو الأنفاق في القاهرة إنهم يعرفوا أحسن طريق يركبوه من محطة لمحطة.

إحنا عندنا 3 خطوط مترو: - خط 1 (الأحمر): من حلوان للمرج الجديدة — 35 محطة - خط 2 (الأزرق): من شبرا الخيمة للمنيب — 20 محطة - خط 3 (الأخضر): من عدلي منصور لجامعة القاهرة — 34 محطة
المجموع: 89 محطة.

المشكلة: المستخدم محتاج يعرف إزاي يروح من محطة لمحطة — يركب إيه، ينزل فين، يتغير فين، ويدفع كام.

الحل: الـ Application بيحسب: 1. أقصر طريق بين المحطتين (أقل عدد محطات) 2. سعر التذكرة (حسب القواعد: $8 - 9 = 1$ جنيه، $10 - 16 = 17$ جنيه، $15 = +17$ جنيه) 3. وقت الرحلة (دقيقتين لكل محطة) 4. عدد مرات تغيير الخط (Transfer)

الـ Architecture بتاعتنا: المشروع معمول بـ **Metro-3** - **Tier Architecture** (الـ Web Layer (Controllers, Views, الـ **Metro.Core** - ViewModels) - **Metro.Data** - Business Layer (Entities, Services, Interfaces, DTOs) - **Metro** - Layer (DbContext, Repositories, Migrations, Seed Data)

MVC Pattern: Models (Station, Line, StationConnection, PricingRule). Views (.cshtml files). استخدمنا **Controllers** (HomeController, RoutesController).

إزاي الـ Request بيشتغل: المستخدم يفضل الصفحة، يختار محطتين من Dropdowns، ويضغط Find Route.

الـ POST Request يروح على **RoutesController.Index()**.

الـ Controller ياستدعي **_metroService.GetRouteAsync(fromId, toId)**.

الـ **MetroService** (وده Orchestrator Service) بيشتغل مع: 1. **IRouteService** — اللي عنده **Dijkstra Algorithm** 2. **IGraphBuilder** — اللي بيبنى Graph من Stations و 3 Connections. **ITravelTimeService** — بيحسب الوقت (عدد المحطات 2×4). **IPricingService** — بيحسب السعر من الـ 5 Pricing Rules. **ITransferDetectionService** — بيعد تغييرات الخط

الـ Dijkstra Algorithm: الـ GraphBuilder بيحسب كل المحطات والوصلات من الـ Database وبيبنى Adjacency Dictionary **Dictionary<int, List<Neighbor>>**. وبيخزنه في Memory Cache لمدة 30 دقيقة.

الـ RouteService يشغل Dijkstra على الـ Graph ده باستخدام **PriorityQueue<int, int>** (Min-Heap). بيدأ من محطة البداية ويفحص الجيران لحد ما يوصل للمحطة النهائية.

الـ Database: عندنا 4 Tables في **Lines** - SQL Server: الـ 3 خطوط بألوانهم - **Stations:** 89 محطة بإحداثياتهم - **StationConnections:** الوصلات بين المحطات - **PricingRules:** قواعد التسعير

فيه 5 محطات تحويل (Transfers) بين الخطوط: سادات، شهداء، عتبة، ناصر، جامعة القاهرة.

الـ Frontend: الـ View بتستخدم: - Dropdowns - **Tom Select** قابلة للبحث - **Leaflet** — خريطة تفاعلية من **GPS** - OpenStreetMap — الـ Geolocation API عشان تلاقي أقرب محطة - **Haversine Formula** — تحسب المسافة بين إحداثيات المستخدم والمحطات

الـ Validation: Manual Validation في الـ Controller — بيتأكد إن المحطتين مش فاضيين ومش نفس الحاجة.

CSRF - HTTPS Redirection - Error من يحمي من `[ValidateAntiForgeryToken]` **Security:** - RoutesController ال Handler

ال ضعف في المشروع: 1. ال HomeController مالهوش `[ValidateAntiForgeryToken]` 2. ال RoutesController مستبعد من ال Compilation في ال 3. csproj! ال Validation Manual — مفيش Controllers 4. Data Annotations الاتنين بيعملوا نفس الحاجة — تكرار

طب لو الدكتور سألك على Dijkstra: إحنا استخدمنا Dijkstra عشان ال Shortest Path. ال Graph بيتخزن ك Dictionary عشان ال Lookup سريع. ال PriorityQueue بيضمن إنا دايماً نشوف أقرب محطة. وطبعاً بنستخدم `AsNoTracking` عشان ال Graph Read-only.

طب لو سألك على EF Core: أهم حاجة استخدمناها: `Include` لل Eager Loading، `AsNoTracking` للقراءة، `ValueGeneratedNever` عشان Ids جاهزة من JSON.

ال DI Container: كل حاجة مسجلة ك Scoped — يعني واحدة لكل Request. خدمة ILineRepository مسجلة بس مش مستخدمة (Dead Code).

خلاصة: المشروع ده بيحل مشكلة حقيقية لمستخدمي المترو. بيستخدم Dijkstra عشان أقصر طريق، ويحسب السعر والوقت والتغييرات. معمول بـ Clean Architecture مع 3 Tiers و Repository Pattern و DI. فيه بعض نقاط الضعف اللي ممكن نتكلم عنها في المناقشة.

15. مراجعة ليلة المناقشة

يا صاحبي، ده اللي محتاج تذاكره قبل المناقشة بـ 15-20 دقيقة

Architecture

- **3-Tier:** Metro (Web) → Metro.Core (Business) → Metro.Data (Data)
- **Patterns:** MVC, Repository, DI, Service Layer, DTO
- **Target:** .NET 8.0, SQL Server, EF Core 8

JI Controllers (2)

Controller	JI Actions	الحكم
RoutesController	GET/POST Index	✓ أحسن — ViewModels + MetroService + Anti-Forgery
HomeController	GET/POST Index, Privacy, Error	✗ أقدم — ViewBag + مفيش Anti-Forgery

4 JI Tables

الجدول	فيه إيه
Lines	خطوط (أحمر، أزرق، أخضر) 3
Stations	محطة بإحداثياتها 89
StationConnections	وصلة بين المحطات 182
PricingRules	قواعد: 9-1 = 8, 10 = 16-10, 17 = +15 جنيه 3

محطات التحويل (5)

- سادات: خط 1 (19) ↔ خط 2 (46)
- شهداء: خط 1 (22) ↔ خط 2 (43)
- عتبة: خط 2 (44) ↔ خط 3 (74)
- ناصر: خط 1 (20) ↔ خط 3 (75)
- جامعة القاهرة: خط 2 (50) ↔ خط 3 (89)

بيشغل Dijkstra إزاي ال

```
Dictionary<int, List<Neighbor>> graph
PriorityQueue<int, int>
distances[start] = 0
loop: dequeue smallest → check neighbors → update distances
reconstruct path from end to start using previous dictionary
reverse path
```

EF Core أهم حاجة في ال

الميزة	إستعملناها فين
<code>.Include()</code>	Station مع Line عشان نجيب
<code>.AsNoTracking()</code>	Graph — Read-only قيود
<code>.Select()</code> projection	Ids عشان نجيب بس
<code>ValueGeneratedNever()</code>	JSON من Ids — Entities كل ال
<code>DeleteBehavior.Restrict</code>	Foreign Keys كل ال

الأساسية LINQ Queries ال 7

```
query 1 : .Include(s => s.Line).FirstOrDefaultAsync(s => s.Id == id)
query 2 : .Include(s => s.Line).ToListAsync()
query 3 : .Where(s => s.LineId == lineId).OrderBy(s => s.Order).ToListAsync()
query 4 : .AsNoTracking().Select(s => new StationGraphNode { ... }).ToListAsync()
query 5 : .AsNoTracking().Select(c => new StationConnectionGraphEdge { ... }).ToListAsync()
query 6 : .Where(sc => sc.FromStationId == id || sc.ToStationId == id).ToListAsync()
query 7 : .OrderBy(r => r.MinStations).ToListAsync()
```

ال DI Chain

```
RoutesController → IMetroService → IRouteService → IGraphBuilder
                  → ITravelTimeService
                  → IPricingService → IPricingRuleRepository
                  → ITransferDetectionService
                  → IStationRepository
                  → IPricingRuleRepository
```

Validation

- **Manual** — Data Annotations مش
- `ModelState.IsValid` عشان مفيش — **مش مستخدم** [Required]
- Client-side JavaScript و HTML5 `required`

Security

- RoutesController: ☒ Anti-Forgery
- HomeController: ☒ Anti-Forgery مفيش
- HTTPS + HSTS ☒
- Users (عام — مفيش) Authentication مفيش

نقاط الضعف اللي الدكتور هيضرب فيها

1. **RoutesController** مش متجمع في ال `csproj`: `<Compile Remove>`
2. **HomeController** مفيهوش Anti-Forgery
3. مفيش `ModelState.IsValid`
4. **Controllers** الاتنين بيعملوا نفس الحاجة
5. **Namespace** ال `RouteService` و `GraphBuilder` بس هما في `Metro.Data.Services` في `Metro.Core` — غلط
6. **LineRepository** مش مستخدم — Dead Code
7. **Empty Migrations** — ال `SeedData` و `InitialCreate` فاضيين

أسئلة أكيدة

س: إيه المشروع بتاعك؟ ج: "Cairo Metro Route Planner — بيحسب أقصر طريق بين محطات المترو ويبحدد السعر والوقت."

س: إيه ال Architecture؟ ج: "MVC-3: Metro (Web), Metro.Core (Business), Metro.Data (Data). Tier — Pattern."

س: إزاي ال Dijkstra بيشتغل؟ ج: "PriorityQueue + Adjacency Dictionary. نبدأ من Start Station، نشوف الجيران، نحدث المسافات، نكرر لحد ما نوصل."

س: إيه ال Relationship بين Station و Line؟ ج: "Many-to-One". كل محطة على خط واحد. الخط عنده كتير محطات.

س: ليه ValueGeneratedNever() ؟ ج: "عشان ال Ids جاية من JSON جاهزة."

س: إزاي ال Transfers بتتسبب؟ ج: "بنقارن LineId لكل محطة مع اللي قبلها — لو اختلف، يبقى فيه Transfer".

س: إيه ال DI وليه بنستخدمها؟ ج: "عشان Loose Coupling و Testability. بدل ما ال Controller يعمل new، هو يطلب Service وال Container يديهوله."

س: إيه ال Weak Points في المشروع؟ ج: "1) RoutesController مستبعد من ال Controllers 2) Compile. 3) HomeController Validation Manual. 4) Anti-Forgery مفهوش".

تمام يا معلم — أنت جاهز للمناقشة.

ربنا معاك واجري كويس.

End of PROJECT_DEFENSE_MASTER_GUIDE.md